

Kargo İşletme Sistemi: Kocaeli İlçelerinden KOU Umuttepe’ye Yük ve Rota Planlama

Farahnozkhon Dilovarovna MASUMOVA ve Hayrunisa KORKULU

Kocaeli Üniversitesi

Bilgisayar Mühendisliği Bölümü

Yazılım Laboratuvarı I – Proje III Raporu

2025–2026 Güz Dönemi

Özet—Bu proje kapsamında, Kocaeli’nin ilçe merkezlerinden Kocaeli Üniversitesi (Umuttepe) yerleşkesine taşınacak kargolar için web tabanlı bir *kargo işletme sistemi* geliştirilmiştir. Sistem; kullanıcıların kargo talebi oluşturabildiği, yöneticinin (admin) istasyon ekleyebildiği ve seçilen gün için kargo yükleme ile araç rota planlamasını çalıştırabildiği iki panelli bir mimari sunmaktadır. Backend katmanı Django REST Framework ile geliştirilmiş; kimlik doğrulama JSON Web Token (JWT) tabanlı olarak sağlanmıştır. Frontend katmanı React ve Vite ile tek sayfa uygulaması (SPA) olarak geliştirilmiş; istasyonlar ve araç rotaları Leaflet harita bileşenleri üzerinde görselleştirilmiştir.

Rota çizimi “kuş uçuşu” yaklaşımıyla yapılmamış; yol ağı, OpenStreetMap verisinden üretilmiş Kocaeli sürüş grafi (`graphml`) üzerinden en kısa yol hesaplanarak parça parça güzergâh poligonları (GeoJSON) olarak kaydedilmiş ve haritada çizdirilmiştir. Planlama tarafında iki problem desteklenmiştir: (i) Belirli sayıda araç (**FIXED**) modu, (ii) Sınırsız araç (**UNLIMITED**) modu (gerektiğinde 500 kg kiralık araç eklenir, kiralama maliyeti 200 birim). Her iki modda da hedef fonksiyon **MIN_COST** temelinde çalışmakta; ek olarak **MIN_COST_MAX_COUNT** ve **MIN_COST_MAX_WEIGHT** amaç seçenekleri ile kargo sıralama önceliği değiştirilebilmektedir. Kullanıcı tarafında erişim kontrolü uygulanmış; her kullanıcı yalnızca kendi kargolarının atandığı aracı ve rotayı görebilmektedir.

Anahtar Kelimeler — Django, Django REST Framework, React, Vite, JWT, Leaflet, OpenStreetMap, En kısa yol, Sezgisel optimizasyon, Rota planlama.

I. Giriş

Kargo lojistiğinde maliyeti belirleyen temel unsurlar; yakıt/mesafe maliyeti, araç kapasite kullanımı, teslimat noktalarının mekânsal dağılımı ve ihtiyaç halinde kiralık araç kullanımına bağlı sabit maliyetlerdir. Gerçek hayatta bu problem yalnızca “bir yerden bir yere taşıma” olarak değil; *kısıtlar*

(araç kapasitesi, araç sayısı, günlük iş yükü, istasyonların farklı konumlarda olması), *hedefler* (maliyeti düşürme, mümkün olduğunca çok kargoyu taşıma, ağır kargoları önceliklendirme) ve *operasyonel süreçler* (kargo talebi oluşturma, iptal, planlama tekrar çalıştırma, raporlama) ile birlikte ele alınır. Bu nedenle kargo planlama sistemlerinde hem yazılım mimarisi hem de rota/yükleme algoritmaları birlikte tasarlanmalıdır.

Bu projede Kocaeli’nin ilçe merkezleri “istasyon” olarak modellenmiş; seçilen gün için istasyonlarda biriken kargoların **Kocaeli Üniversitesi Umuttepe Kampüsü**’ne taşınması hedeflenmiştir. Problem iki temel karar adımından oluşur: (i) hangi kargoların hangi araca yükleneceği (*yükleme/atama problemi*), (ii) her araç için hangi istasyonların hangi sırayla ziyaret edileceği (*rota sıralama problemi*). Bu iki alt problem, literatürde *Vehicle Routing Problem (VRP)* ve *bin packing / yükleme* problemleriyle yakından ilişkilidir ve büyük veri boyutlarında doğrudan tüm olasılıkları denemek (brute-force) pratik değildir. Bu sebeple uygulamada, kısa sürede çözüm üreten sezgisel (heuristic) yaklaşımlar kullanılmış ve hesaplama maliyeti düşük tutulmuştur.

Projede yalnızca “kuş uçuşu” mesafe ile çizilen soyut rotalar üretilmemiş; rota çizimi gerçek yol ağını takip edecek biçimde tasarlanmıştır. Bu kapsamda yol ağı, OpenStreetMap verilerinden çıkarılmış `kocaeli_drive.graphml` sürüş grafi üzerinden işlenmiş; istasyonlar arası en kısa yol hesaplanarak her iki durak arasındaki güzergâh **GeoJSON polylines** biçiminde parça parça saklanmış ve Leaflet üzerinde harita katmanı olarak gösterilmiştir. Böylece kullanıcı/admin haritada gördüğü rotayı gerçekçi biçimde yorumlayabilmekte; özellikle

uzak ilçelerde “en yakın gibi görünen” fakat yol ağı açısından daha uzun sürebilecek güzergâhlar doğru şekilde temsil edilebilmektedir.

Ödev isterleri doğrultusunda sistemde iki panel yer almaktadır:

- **Kullanıcı Paneli:** Kullanıcılar istasyon seçerek kargo talebi (ağırlık, adet, tarih) oluşturabilmekte, planlama öncesinde talebini iptal edebilmekte ve planlama çalıştırıldıktan sonra yalnızca kendisine ait kargoların yüklendiği aracın rotasını görebilmektedir. Bu tasarım, hem gizlilik hem de erişim kontrolü açısından gereklidir.
- **Yönetici (Admin) Paneli:** Admin istasyon ekleyebilmekte/güncelleyebilmekte, belirli bir gün için planlamayı çalıştırabilmekte, her araç için toplam mesafe ve maliyet özetini görebilmekte ve tüm araç rotalarını tek ekranda haritada inceleyebilmektedir. Ayrıca araç detayında hangi kargonun hangi kullanıcıya ait olduğu gibi yönetsel raporlama ihtiyaçları karşılanmaktadır.

Sistem, yeniden planlama gereksinimi de göz önünde bulundurularak tasarlanmıştır: Aynı gün için yeni talepler geldiğinde veya bazı talepler iptal edildiğinde admin planlamayı tekrar çalıştırabilir; her çalıştırma bir “sefer” olarak kaydedilir ve geçmiş seferler listelenebilir. Bu yaklaşım, gerçek operasyonlarda sık görülen “gün içi değişiklik” durumlarına uyum sağlar.

Bu rapor kapsamında; geliştirilen sistemin gereksinimleri ve kapsamı, istemci-sunucu mimarisi, veritabanı ve API tasarımı, planlama algoritmasının sezgisel yaklaşımı, gerçek yol ağı üzerinden rota üretimi ve örnek senaryolara göre deneysel sonuçları ayrıntılı olarak sunulacaktır. Ayrıca kapasite yetersizliği, kiralık araç kullanımı ve ağır kargo senaryoları gibi sınır durumlarda sistemin verdiği tepkiler tartışılarak genel değerlendirme yapılacaktır.

II. Yöntem ve Sistem Mimarisi

Bu bölümde uygulamanın yazılım mimarisi, veri modeli, API katmanı, güvenlik (JWT), yetkilendirme kısıtları ve harita/rota üretim tekniği ayrıntılı biçimde açıklanmaktadır. Sistem, istemci-sunucu yaklaşımıyla geliştirilmiş; iş mantığı ve veri erişimi backend katmanına, kullanıcı etkileşimi ve görselleştirme ise frontend katmanına taşınmıştır. Böylece

hem ölçeklenebilirlik hem de bakım/ek geliştirme kolaylığı sağlanmıştır.

A. Genel Mimari

Sistem iki ana bileşenden oluşur:

- **Backend (Django REST API):** Django REST Framework üzerinde modüler uygulamalar şeklinde kurgulanmıştır. `stations`, `cargo`, `planning`, `routing`, `accounts` uygulamaları sorumluluklarına göre ayrıştırılmıştır. Tüm veri alışverişi JSON formatındadır ve JWT doğrulama ile korunan REST uç noktaları üzerinden sağlanır.
- **Frontend (React + Vite):** Kullanıcı ve admin arayüzleri SPA olarak geliştirilmiştir. Sayfalar `react-router-dom` ile yönetilir; API iletişimi `axios` üzerinden yapılır. Harita katmanında Leaflet kullanılarak istasyonlar, durak sırası ve rota segmentleri görselleştirilir.

Bu ayırım sayesinde; (i) backend tarafında planlama/rota algoritmaları bağımsız geliştirilmiş, (ii) frontend tarafında aynı API üzerinden hem kullanıcı hem admin senaryoları tek kod tabanı ile yönetilmiştir.

B. Backend: Uygulamalar, Modeller ve API Uç Noktaları

Backend katmanında DRF’in `serializer-view-url` düzeni kullanılmış; kimlik doğrulama, izinler ve standart hata yanıtları tutarlı hale getirilmiştir. Özellikle `settings.py` üzerinde:

- **JWT doğrulama:** `rest_framework_simplejwt` ile erişim/yenileme token yapısı kullanılır.
- **Permissions:** Varsayılan olarak `IsAuthenticated`; admin uç noktalarında `IsAdminUser`.
- **CORS:** React uygulamasının farklı porttan istek atabilmesi için CORS izinleri yapılandırılır.
- **Pagination:** Liste dönen uç noktalarda sayfalama aktif edilerek hem sunucu yükü hem istemci render maliyeti düşürülür.

1) *Stations Uygulaması (stations):* **Amaç:** Kocaeli ilçeleri ve KOU Umuttepe gibi sabit noktaların sistemde istasyon olarak tutulması.

Model: Station temel alanları:

- `name`: İlçe adı / istasyon adı,
- `lat, lng`: coğrafi koordinatlar,

- `is_active`: istasyonun planlamaya dahil edilip edilmeyeceği,
- (opsiyonel) `created_at`, `updated_at`: izlenebilirlik.

API davranışı:

- Kullanıcılar sadece aktif istasyonları listeleyebilir (GET /api/stations/).
- Admin istasyon ekleyebilir/güncelleyebilir/silebilir (CRUD).
- Planlama sırasında istasyon koordinatları rota segment üretiminde doğrudan kullanılır.

2) *Cargo Uygulaması (cargo)*: **Amaç:** Kullanıcıların belirli bir gün için istasyondan alınacak kargo talebi oluşturmaları.

Model: CargoRequest temel alanları:

- `user` (FK): talep sahibi,
- `station` (FK): kargonun bulunduğu istasyon,
- `total_weight` (kg), `count`: talep büyüklüğü,
- `planned_date`: hangi gün taşınacak,
- `status`: PENDING / PLANNED / REJECTED / CANCELED,
- `assigned_vehicle` (FK, nullable): planlama sonucu atanan araç.

İş kuralları:

- Kullanıcı yalnızca kendi kargolarını listeler/görür.
- `cancel` aksiyonu sadece PENDING durumunda çalışır; planlama sonrasında iptal kısıtlanır.
- Aynı kullanıcı aynı gün farklı istasyonlardan birden çok talep girebilir; planlama bunu topluca ele alır.

3) *Planning Uygulaması (planning)*: **Amaç:** Seçili gün için kargoların araçlara atanması, durak sırası çıkarılması, gerçek rota segmentlerinin üretilmesi ve raporlanması.

Ana modeller:

- `PlanningRun`: Bir planlama çalıştırmasını (seferi) temsil eder. Alanlar: `planned_date`, `mode` (FIXED/UNLIMITED), `objective`, `km_cost`, `rental_vehicle_cost`, `rental_vehicle_capacity`, `is_active`.
- `VehicleRoute`: Her seferde her araç için bir rota özeti tutar. Alanlar: `capacity`,

`used_weight`, `total_km`, `total_cost`, `is_rental`.

- `RouteStop`: Bir aracın ziyaret edeceği durakları sıralı tutar (stop index).
- `RouteSegment`: Duraklar arasındaki gerçek yol geometrisini (GeoJSON) ve segment km değerini tutar.

Sefer tekrar çalıştırma yaklaşımı: Aynı tarih için yeni bir planlama çalıştırıldığında önceki sefer `is_active=False` yapılarak arşivlenir, ilgili tarihteki kargolar tekrar PENDING durumuna çekilir ve yeni sefer üretilir. Bu, ödevde istenen “sistem tekrar çalıştırılabilir” gereksinimini karşılar.

4) *Routing Uygulaması (routing)*: **Amaç:** Kuş uçuşu çizim yerine gerçek yol ağı ile tutarlı rota üretmek.

Veri kaynağı: OpenStreetMap verisinden türetilen Kocaeli sürüş grafi `kocaeli_drive.graphml` formatında projeye dahil edilmiştir. Graf düğümleri yol kesişim/bağlantı noktalarını; kenarlar ise sürüşe açık yol segmentlerini temsil eder. Her kenarda `length` alanı (km/metre) ağırlık olarak kullanılır.

Hesaplama adımı: Her iki durak koordinatı graf üzerinde en yakın düğüme eşleştirilir; ardından Dijkstra tabanlı en kısa yol ile düğüm dizisi çıkarılır. Bu düğüm dizisi üzerinden GeoJSON `LineString` üretilerek `RouteSegment.polyline_geojson` alanına kaydedilir. Böylece frontend aynı segmenti doğrudan çizerek yol geometrisini görür.

5) *Accounts / Güvenlik (accounts)*: **Kimlik doğrulama:** Kullanıcı kayıt/giriş işlemleri sonucunda `access/refresh` token üretilir. Frontend her istekte `Authorization: Bearer <token>` başlığı gönderir.

Yetkilendirme:

- Admin: istasyon CRUD, planlama çalıştırma, tüm seferleri ve tüm araç rotalarını görme.
- Kullanıcı: sadece kendi kargo talepleri, sadece kendi kargolarının atandığı rota detayları.

Bu kısıtlar, kullanıcılar arasında veri sızıntısını engeller ve ödevde istenen kullanıcı görünürlüğü şartını sağlar.

C. API Tasarımı: Uç Noktalar ve Yanıt Yapısı

API uç noktaları fonksiyonel alanlara göre gruplanmıştır:

- /api/auth/register/, /api/auth/token/, /api/auth/me/
- /api/stations/ (GET kullanıcı, CRUD admin)
- /api/cargo-requests/ (kullanıcıya özel; create/list/cancel)
- /api/planning/run/ (POST, admin: planlama çalıştır)
- /api/planning/runs/ (GET, admin: sefer listesi)
- /api/planning/runs/{id}/vehicles/ (GET, admin: tüm araç rotaları)
- /api/planning/vehicles/{id}/detail/ (GET, admin: araç + kargo sahipleri)
- /api/planning/my-route/?date=YYYY-MM-DD (GET, kullanıcı: sadece kendi rotası)

Yanıtlar standart JSON şemasıyla döner: liste uç noktalarında sayfalama (count/next/previous/results), detay uç noktalarında ise araç özeti + durak listesi + segment listesi birlikte döndürülür. Örneğin kullanıcı rotasında frontend'in haritayı çizebilmesi için RouteStop ve RouteSegment verileri aynı yanıt içerisinde sağlanır.

D. Frontend: Sayfalar, Bileşenler ve İstemci İçi Akış

Frontend katmanı SPA yapısındadır ve temel olarak: (i) oturum yönetimi, (ii) form tabanlı kargo oluşturma, (iii) admin planlama parametre ekranı, (iv) harita görselleştirme üzerine kuruludur.

1) Sayfalar:

- **Login/Register:** Token alımı ve saklanması (localStorage).
- **Kullanıcı Paneli:** Kargo oluşturma formu (istasyon, tarih, kg, adet), talepler tablosu, iptal butonu.
- **Benim Rotam:** Tarih seçimi ile backend'den sadece ilgili kullanıcıya ait rota çekilir ve haritada çizilir.
- **Admin Paneli:** Planlama parametreleri (tarih, FIXED/UNLIMITED, objective), "planlamayı çalıştır" aksiyonu.
- **Sefer Detayı:** Araç kartları (km, maliyet, kapasite kullanımı), araç seçimi ile durak/segmentlerin haritada gösterimi.

- **İstasyon Yönetimi:** Admin için istasyon ekleme/güncelleme formları.

2) Axios Katmanı ve Token Entegrasyonu:

axios üzerinde baseURL belirlenmiş; *request interceptor* ile access token otomatik eklenmiştir. 401 durumlarında kullanıcı login sayfasına yönlendirilerek korumalı sayfalar güvence altına alınmıştır. Bu yaklaşım hem kod tekrarını azaltmış hem de güvenlik politikasını merkezileştirmiştir.

3) *Harita Bileşenleri (Leaflet):* Harita ekranında üç görsel katman birlikte kullanılır:

- **İstasyon Marker'ları:** İlçe merkezleri ve KOU Umutepe farklı ikon/reng ile işaretlenebilir.
- **Durak Sırası:** RouteStop.order alanına göre numaralı marker/tooltip gösterilir (1,2,3,...).
- **Rota Segmentleri:** RouteSegment.polyline_geojson içindeki koordinatlar polyline'a dönüştürülerek çizilir.

Böylece kullanıcı ve admin aynı veri yapısıyla hem rotanın sırasını hem de yol geometrisini tutarlı şekilde görebilir.

E. Veri Akışı Özeti

Sistemin tipik veri akışı şu şekilde özetlenebilir:

- 1) Kullanıcı istasyon/tarih/ağırlık/adet girerek kargo talebi oluşturur (PENDING).
- 2) Admin seçili gün için planlamayı çalıştırır; kargolar araçlara atanır.
- 3) Her araç için durak sırası çıkarılır; duraklar arası OSM grafindan en kısa yol segmentleri hesaplanır.
- 4) Araç özetleri, duraklar ve segmentler veritabanına yazılır; kargolar PLANNED veya REJECTED olur.
- 5) Kullanıcı "benim rotam" ekranında yalnızca kendi kargolarının bulunduğu araç(lar)ın rotasını görür; admin tüm rotaları görür.

III. Planlama Algoritması ve Yalancı Kod

Bu bölümde sistemin çekirdek planlama akışı, projedeki gerçek Django kod akışına sadık kalınarak **yalancı kod** (pseudocode) biçiminde sunulmaktadır. Amaç; hem **hangi adımların yapıldığını** (algoritma) hem de bu adımların **kod benzeri ama dilden bağımsız** anlatımını (yalancı kod) netleştirmektir.

A. Algoritma ve Yalancı Kod Ayrımı (Kısa Not)

Algoritma: Bir problemi çözmek için izlenen adımların mantıksal tarifidir (ör. “kargoları sırala, araca ata, durakları sırala, maliyeti hesapla”).

Yalancı kod (pseudocode): Algoritmayı, gerçek koda çok benzer şekilde ama Python/Django sözdizimine birebir bağlı olmadan anlatan gösterimdir. Bu raporda her “Algoritma” başlığı altında adımlar yalancı kod biçiminde listelenmiştir.

B. Problem Tanımı

Sistem seçilen gün (`planned_date`) için PENDING durumundaki kargoları alır, araçlara atar, her araç için durak sırası çıkarır ve OSM yol ağı üzerinde en kısa yol segmentlerini üreterek veritabanına yazar.

FIXED modu: Başlangıçta 3 araç vardır ve kiralama yoktur. Kapasiteler varsayılan olarak {500, 750, 1000} kg’dır. Kapasite yetmezse bazı kargolar REJECTED olur.

UNLIMITED modu: Başlangıç yine {500, 750, 1000} kg araçlarla başlar; ihtiyaç olursa 500 kg kapasiteli **kiralık** araç eklenebilir. Kiralık araç sabit maliyeti 200 birimdir.

Kodda varsayılanlar: `km_cost=1, rental_vehicle_cost=200, rental_vehicle_capacity_kg=500.`

C. Amaç (Objective) Seçenekleri

Kodda kargo sıralama (önceliklendirme) aşağıdaki gibidir:

- **MIN_COST:** `total_weight_kg` azalan sırada.
- **MIN_COST_MAX_COUNT:** `cargo_count` azalan sırada.
- **MIN_COST_MAX_WEIGHT:** `total_weight_kg` azalan sırada.

D. Çekirdek Akış: Admin Planlama Çalıştırma

Aşağıdaki akış, backend’deki `/api/planning/run/` (POST, admin) endpoint’inin yaptığı işlemleri birebir özetler.

Algoritma 1: Sefer Oluşturma ve Ön Hazırlık (`/api/planning/run/`)

- 1) Girdi: `planned_date, mode, objective` (opsiyonel olarak `km_cost, rental_vehicle_cost,`

`rental_vehicle_capacity_kg, base_vehicle_capacities`).

- 2) `planned_date` yoksa hata döndür.
- 3) `mode` sadece `FIXED` veya `UNLIMITED` değilse hata döndür.
- 4) `objective` üç seçenekten biri değilse hata döndür.
- 5) KOU istasyonunu bul:
 - `name` alanı “Kocaeli Üniversitesi (Umut-tepe)” (ASCII varyantı dahil).
 - Yoksa hata döndür (seed bekleniyor).
- 6) Aynı tarihte aktif olan eski seferleri pasifleştir: `PlanningRun(planned_date=..., is_active=True) → is_active=False.`
- 7) Yeniden planlanabilirlik için o tarihteki tüm kargoları sıfırla:
 - `status=PENDING`
 - `assigned_vehicle_id=NULL`
 - `assigned_run_id=NULL`
- 8) Yeni `PlanningRun` kaydı aç ve parametreleri kaydet.
- 9) Başlangıç araçlarını oluştur:
 - Her cap için `VehicleRoute(capacity_kg=..., is_rented=False)` oluştur.
 - `used_weight_kg=0, used_count=0, total_km=0, total_cost=0.`
- 10) O günün PENDING kargolarını çek (`CargoRequest.requested_date=planned_date`).
- 11) Kargoları `objective`’e göre sırala (Algoritma 2).
- 12) Kargoları araçlara ata (Algoritma 3–4).
- 13) Her araç için durak sırası ve segmentleri üret (Algoritma 5–7).
- 14) Çıktı: `run_id, arac_sayisi, reddedilen_kargo_sayisi.`

E. Kargo Sıralama

Algoritma 2: Kargoları Objective’e Göre Sırala

- 1) Girdi: `cargos, objective.`
- 2) Eğer `objective=MIN_COST_MAX_COUNT` ise: `cargos` listesini `cargo_count` azalan sırada sırala.
- 3) Değilse (`MIN_COST` veya `MIN_COST_MAX_WEIGHT`): `cargos`

listesini `total_weight_kg` azalan sırada sırala.

4) Çıktı: sıralanmış `cargos`.

F. Kargo Atama: Greedy (Sezgisel) Yaklaşım

Koddaki ana fikir: Her kargo için tüm uygun araçlara bakılır, “tahmini ek maliyeti” en düşük olan araç seçilir. Tahminde **Haversine** kullanılır; gerçek yol daha sonra OSM grafindan çıkarılır.

Algoritma 3: Araç Uygunluğu ve Tahmini Ek Mesafe

- 1) **Kapasite kontrolü:** $(used_weight_kg + cargo.total_weight_kg) \leq capacity_kg$ ise sığar.
- 2) **Tahmini ek km (Δkm) hesabı:**

 - Araç henüz hiç istasyon almamışsa: $\Delta km \approx 2 \times d(KOU, station)$.
 - Araçta istasyonlar varsa: yeni istasyonun, araçtaki mevcut istasyonlara olan en yakın mesafesini bul: $nearest = \min d(s_i, new)$. Yaklaşık ek: $\Delta km \approx 2 \times nearest$.
 - Eğer yeni istasyon zaten araç setindeyse: $\Delta km = 0$.

- 3) **Tahmini ek maliyet:** $\Delta cost = \Delta km \times km_cost$.

Algoritma 4: Kargoyu Araca Yerleştirme (Greedy Atama Döngüsü)

- 1) Her kargo c için:
 - a) Eğer $c.station$ boş/eksikse: REJECTED yap ve devam et.
 - b) $best_vehicle=$ None, $best_extra_cost=$ None.
 - c) Her araç v için:
 - Eğer kapasite sığmıyorsa atla.
 - $\Delta cost$ hesapla (Algoritma 3).
 - En küçük $\Delta cost$ üreten aracı $best_vehicle$ olarak seç.
 - d) Eğer $best_vehicle$ yoksa:
 - FIXED ise: $c \rightarrow$ REJECTED.
 - UNLIMITED ise: **yeni kiralık araç aç** ($capacity=rental_vehicle_capacity_kg$). Eğer kargo sığmıyorsa kiralığa ata, sığmıyorsa REJECTED.
 - e) Eğer $best_vehicle$ varsa:

- UNLIMITED modunda, eğer $best_extra_cost > rental_vehicle_cost$ ise:
 - Yeni kiralık araç aç.
 - Kargo kiralığa sığmıyorsa kiralığa ata.
 - Sığmıyorsa (koddaki davranış): $best_vehicle$ ’a ata.
- Aksi durumda: $best_vehicle$ ’a ata.

2) Bu atama sırasında yapılan DB güncellemeleri:

- Atanınca: $status=PLANNED$, $assigned_vehicle_id=v.id$, $assigned_run_id=run.id$.
- Araç güncelle: $used_weight_kg += cargo.weight$, $used_count += cargo.count$.
- Reddedilince: $status=REJECTED$, $assigned_vehicle_id=NULL$, $assigned_run_id=run.id$.

G. Durak Sıralama: Yakın Komşu Sezgiseli

Koddaki `greedy_order_from_depot` fonksiyonu, durakları KOU’dan başlayarak yakın komşu mantığı ile sıralar (Haversine mesafesiyle).

Algoritma 5: Durakları Sırala (Nearest Neighbor)

- 1) Girdi: KOU (depot), aracın tekil istasyon listesi $\{s_1, \dots, s_k\}$.
- 2) $remaining=$ depot olmayan istasyonlar.
- 3) $cur=$ depot koordinatı.
- 4) $ordered$ boş liste.
- 5) $remaining$ bitene kadar:
 - $nxt=$ $remaining$ içinden cur ’a en yakın istasyon (Haversine).
 - $ordered$ listesine ekle, $cur=nxt$, $remaining$ ’den çıkar.
- 6) Çıktı: $ordered$ (KOU hariç sıralı duraklar).

H. Gerçek Yol: OSM Grafi Üzerinden En Kısa Yol Segmentleri

Bu aşamada artık Haversine değil, `kocaeli_drive.graphml` sürüş grafi üzerinden NetworkX shortest-path ile segmentler üretilir ve `RouteSegment.polyline_geojson` alanına yazılır.

Algoritma 6: Her Araç İçin Stop ve Segmentleri Üret (RouteStop, RouteSegment)

- 1) Her araç v için, o araca atanmış kargolardan tekil istasyonları çıkar.
- 2) Eğer hiç istasyon yoksa:
 - $total_km=0$
 - $total_cost = (is_rented ? rental_vehicle_cost : 0)$
 - devam et
- 3) Durakları sırala: $district_stops = greedy_order_from_depot(KOU, stops)$.
- 4) Tam stop listesi oluştur: $full_stops = [KOU] + district_stops + [KOU]$.
- 5) **Stop kayıtları:** $stop_order=1..n$ olacak şekilde RouteStop kayıtlarını yaz.
- 6) **Segment üretimi:** ardışık stop çiftleri için:
 - $a = stop[i], b = stop[i+1]$
 - $(geojson, km) = en_kisa_yol_geojson_ve_mesafe(a, b)$
 - $RouteSegment(from_order=a, order, to_order=b.order, distance_km=km, polyline_geojson=geojson)$
 - $toplam_km += km$
- 7) Araç maliyeti:
 - $toplam_maliyet = toplam_km * km_cost$
 - Eğer kiralıksa: $toplam_maliyet += rental_vehicle_cost$
 - $v.total_km = toplam_km$, $v.total_cost = toplam_maliyet$

Algoritma 7: En Kısa Yol GeoJSON ve Mesafe (en_kisa_yol_geojson_ve_mesafe)

- 1) Eğer graf önbellekte yoksa yükle:
 - $kocaeli_drive.graphml$ varsa dosyadan oku.
 - Yoksa OSM'den Kocaeli sürüş grafi indir, kenar uzunluklarını ekle, graphml olarak kaydet.
- 2) Grafi projeksiyonlu hale getir ($ox.project_graph$).
- 3) Başlangıç/bitiş koordinatları için en yakın düğümleri bul:
 - Noktayı graf CRS'ine projekte et, $nearest_nodes$ ile düğüm seç.

- Hata olursa Transformer ile projekte edip tekrar dene.

- 4) $nx.shortest_path(G, bas, bit, weight="length")$ ile düğüm listesi üret.
- 5) Her ardışık düğüm çifti (u, v) için:
 - Kenar verisini al; birden çok kenar varsa $length$ en küçük olanı seç.
 - $toplam_m += length$.
 - Geometri varsa $geometry.xy$ noktalarını al; yoksa $(node[u], node[v])$ düz çizgi kullan.
 - Koordinat birleştirmede tekrar eden noktayı kırp (son==ilk ise).
- 6) Çıktı:
 - $GeoJSON.LineString(coordinates)$
 - $km = toplam_m/1000$
- 7) Hata yakalanırsa: iki nokta arasında düz çizgi GeoJSON döndür, $km=0$.

I. Erişim ve Gizlilik: Kullanıcıya Sadece Kendi Rotası

Kullanıcı endpoint'i `/api/planning/my-route/?date=YYYY-MM-DD` yalnızca kullanıcıya atanmış araçların stop+segmentlerini döndürür.

Algoritma 8: Kullanıcıya “Benim Rotam” Dön (/api/planning/my-route/)

- 1) Girdi: $date$ (query param).
- 2) $date$ yoksa 400 döndür.
- 3) O günün aktif seferini bul: $PlanningRun(planned_date=date, is_active=True)$ (yoksa 404).
- 4) Kullanıcının o gün için planlanan kargolarını çek: $CargoRequest(user=current, requested_date=date, assigned_run_id=run.id, status=PLANNED, assigned_vehicle_id!=N)$
- 5) Bu kargolardan $vehicle_ids$ kümesini çıkar; boşsa 404 döndür.
- 6) $VehicleRoute$ kayıtlarını getir: $id \text{ IN } vehicle_ids \text{ AND } planning_run=run, stops+segments \text{ prefetch}$.
- 7) Çıktı JSON: $run_id, planned_date$, her araç için $total_km, total_cost, stops, segments$.

J. Kargo Talebi Yönetimi (Kullanıcı Paneli)

Aşağıdaki akışlar, kullanıcıların kargo talebi oluşturma, listelemesi ve iptal etmesi işlemlerinin backend tarafındaki karşılığını yalancı kod biçiminde özetler.

Algoritma 9: Kargo Talebi Oluşturma (/api/cargo-requests/)

- 1) Girdi: station_id, total_weight_kg, cargo_count, (ops.) requested_date.
- 2) JWT doğrulaması yapılır; kullanıcı yoksa 401 döndür.
- 3) station_id ile istasyon bulunur; yoksa 404 döndür.
- 4) Alan doğrulama:
 - total_weight_kg > 0 değilse 400,
 - cargo_count > 0 değilse 400,
 - requested_date yoksa varsayılan olarak “bugün” atanır.
- 5) Yeni CargoRequest kaydı oluştur:
 - user = request.user
 - station = station_id
 - total_weight_kg, cargo_count
 - status = PENDING
 - assigned_run_id = NULL,
 - assigned_vehicle_id = NULL
- 6) Oluşturulan kayıt JSON olarak döndürülür (200/201).

Algoritma 10: Kullanıcının Kargolarını Listeleme (/api/cargo-requests/)

- 1) JWT doğrulaması yapılır; kullanıcı yoksa 401 döndür.
- 2) Sorgu: CargoRequest WHERE user=current_user çek.
- 3) İsteğe bağlı filtreler:
 - ?date= verilmişse o tarihe filtrele,
 - ?status= verilmişse duruma filtrele.
- 4) Kayıtları created_at azalan sırada döndür.

Algoritma 11: Kargo İptal Etme (cancel) (/api/cargo-requests/<id>/cancel/)

- 1) JWT doğrulaması yapılır; kullanıcı yoksa 401 döndür.
- 2) id ile kargo bulunur; yoksa 404 döndür.
- 3) Yetki kontrolü: cargo.user == current_user değilse 403 döndür.
- 4) Eğer cargo.status != PENDING ise 400 döndür (planlanan/ reddedilen iptal edilemez).

5) Kargo durumu “iptal” için güncellenir:

- (Kod davranışına göre) status = REJECTED
- assigned_run_id ve assigned_vehicle_id zaten NULL kalır.

- 6) {success: true} veya güncel kargo JSON’u döndür.

K. Kimlik Doğrulama (JWT) ve Kullanıcı Bilgisi

Bu akışlar, frontend’in giriş/kimlik doğrulama sürecinde kullandığı temel endpoint’leri açıklar.

Algoritma 12: Kullanıcı Kaydı (/api/auth/register/)

- 1) Girdi: username, password, (ops.) email.
- 2) username boşsa 400 döndür.
- 3) username daha önce kullanılmışsa 400 döndür.
- 4) password politika kontrolünden geçmezse 400 döndür.
- 5) Django user oluştur: create_user(username, email)
- 6) Yanıt: kullanıcı temel bilgileri JSON olarak döndür.

Algoritma 13: Token Alma (Login) (/api/auth/token/)

- 1) Girdi: username, password.
- 2) Kullanıcı doğrula; başarısızsa 401 döndür.
- 3) access ve refresh token üret.
- 4) Yanıt: {access, refresh} döndür.

Algoritma 14: Me (Oturum Kullanıcısı) (/api/auth/me/)

- 1) JWT doğrulaması yapılır; token yoksa/yanlışsa 401 döndür.
- 2) request.user serileştirilir (id, username, is_staff vb.).
- 3) Yanıt: kullanıcı bilgileri JSON.

L. İstasyon Yönetimi (Admin + Kullanıcı)

İstasyonlar kullanıcı tarafında sadece okunur; admin tarafında CRUD yapılabilir.

Algoritma 15: İstasyonları Listele (/api/stations/)

- 1) JWT doğrulaması yapılır; kullanıcı yoksa 401 döndür.
- 2) Aktif istasyonları getir: Station WHERE is_active=True.
- 3) Sırala: name artan.

- 4) Yanıt: istasyon listesi (id, name, lat, lon, is_active) JSON.

Algoritma 16: Admin İstasyon Ekle (/api/stations/)

- 1) JWT doğrulaması yapılır; kullanıcı yoksa 401 döndür.
- 2) Admin kontrolü: is_staff değilse 403 döndür.
- 3) Girdi: name, latitude, longitude, (ops.) is_active.
- 4) name normalize edilir (ASCII dönüştürme uygulanır; ör. "Ü"→"U").
- 5) Yeni Station kaydı oluştur ve kaydet.
- 6) Yanıt: oluşturulan istasyon JSON.

Algoritma 17: İstasyon Seed Komutu (Yükleme)

- 1) Sabit ilçe listesi + KOU Umuttepe depo istasyonu tanımlanır.
- 2) Her kayıt için:
 - update\or\create(name, defaults=\{lat, lon, is_active=True\})
- 3) Çıktı: eklenen/güncellenen kayıt sayıları loglanır.

M. Admin İzleme ve Rota İnceleme Akışları

Bu kısım admin panelinin seferleri/araçları/rotaları listeleme ve detay görme akışlarını kapsar.

Algoritma 18: Seferleri Listele (/api/planning/runs/)

- 1) JWT doğrulaması yapılır; kullanıcı yoksa 401 döndür.
- 2) Admin kontrolü yapılır; değilse 403 döndür.
- 3) (Ops.) ?date= verilmişse sadece o güne filtrele.
- 4) Her PlanningRun için:
 - Sefer araçlarını çek: VehicleRoute WHERE planning_run=run.
 - toplam_km=sum(vehicle.total_km).
 - toplam_cost=sum(vehicle.total_cost).
 - arac_sayisi = count(vehicles).
- 5) Yanıt: sefer listesi JSON (özet alanlarla).

Algoritma 19: Seferdeki Araçları ve Rotaları Getir (/api/planning/runs/<id>/vehicles/)

- 1) JWT doğrulaması + admin kontrolü (401/403).
- 2) run_id ile sefer bulunur; yoksa 404 döndür.
- 3) Seferin tüm araçları çekilir (araç no'ya göre sıralı).
- 4) Her araç için:
 - Araç özet alanları: capacity_kg, used_weight_kg, used_count, is_rented, total_km, total_cost.
 - Stoplar: RouteStop WHERE vehicle_route=v ORDER BY stop_order.
 - Segmentler: RouteSegment WHERE vehicle_route=v ORDER BY from_order.
- 5) Yanıt: vehicles:[\{vehicle, stops, segments\}] JSON.

Algoritma 20: Araç Detayı + Kargo Sahipleri (/api/planning/vehicles/<id>/detail/)

- 1) JWT doğrulaması + admin kontrolü (401/403).
- 2) vehicle_id ile VehicleRoute bulunur; yoksa 404.
- 3) Bu araca atanmış kargolar çekilir: CargoRequest WHERE assigned_vehicle_id=vehicle_id AND status=PLANNED.
- 4) Kargolar kullanıcıya göre gruplanır:
 - Her user için toplam_agirlik, toplam_adet hesaplanır.
 - İstasyon bazında alt liste oluşturulur (station_name, weight, count).
- 5) Araç stop/segmentleri eklenir.
- 6) Yanıt: vehicle + owners + route JSON.

Algoritma 21: Admin Özet (Summary) (/api/planning/admin/summary/)

- 1) JWT doğrulaması + admin kontrolü (401/403).
- 2) Toplam planlanan kargolar çekilir: CargoRequest WHERE status=PLANNED.
- 3) Toplamlar hesaplanır:
 - total_cargo_weight = sum(total_weight_kg)

- `total_cargo_count = sum(cargo_count)` veya kayıt sayısı
- `total_vehicles = count(VehicleRoute)`
- `total_runs = count(PlanningRun)`
- `total_distance = sum(RouteSegment.distance_km)`
- `total_cost = sum(VehicleRoute.total_cost)`

4) Yanıt: özet alanları JSON.

N. Frontend Harita Çizimi ile Bağlantılı Veri Akışı (Kısa)

Bu bölüm backend algoritmalarını tamamlamak için, frontend'in haritayı nasıl beslediğini açıklar.

Algoritma 22: Admin Haritada Tüm Rotaları Göster (Frontend Akışı)

- 1) Admin seçilen sefer için `/api/planning/runs/<id>/vehicles/` çağırır.
- 2) Dönen `segments[*].polyline_geojson` alınır.
- 3) Leaflet üzerinde her segment Polyline olarak çizilir.
- 4) `stops[*]` marker olarak numaralandırılır.
- 5) Araçlar farklı kartlarda listelenir; kart tıklanınca sadece o aracın rotası filtrelenebilir.

Algoritma 23: Kullanıcı “Benim Rotam” Harita Akışı (Frontend)

- 1) Kullanıcı tarih seçer ve `/api/planning/my-route/?date=YYYY-MM-DD` çağırır.
- 2) Dönen `routes[*].segments[*].polyline\__geojson` haritada çizilir.
- 3) Kullanıcı sadece kendi `vehicle_ids` rotasını gördüğü için gizlilik korunur.

O. Hata Senaryoları ve Dayanıklılık (Backend)

Bu alt bölüm, sistemin gerçek kullanımda karşılaşılabileceği hata/istisna durumlarını ve koddaki tipik dayanıklılık davranışlarını yalancı kod ile özetler.

Algoritma 24: Yetki Kontrolü (Admin-Only Endpoint Koruması)

- 1) İstek gelen endpoint için `request.user` doğrulanır (JWT).
- 2) Eğer kullanıcı yoksa: 401 Unauthorized.
- 3) Eğer endpoint admin-only ise:
 - `request.user.is_staff == False` ise 403 Forbidden.
 - Aksi halde işlem devam eder.

4) Admin-only örnekleri:

- `/api/planning/run/`
- `/api/planning/runs/`
- `/api/planning/runs/<id>/vehicles/`
- `/api/planning/vehicles/<id>/detail/`
- `/api/planning/admin/summary/`
- `/api/stations/` (POST/PUT/PATCH/DELETE)

Algoritma 25: Zorunlu Parametre Kontrolü (Common Validation)

1) Planlama isteğinde:

- `planned_date` yoksa: 400 Bad Request.
- `mode` tanımsız ise: 400.
- `objective` tanımsız ise: 400.

2) Kargo oluşturma isteğinde:

- `station_id` yoksa: 400.
- `total_weight_kg <= 0` ise: 400.
- `cargo_count <= 0` ise: 400.

3) my-route isteğinde:

- `date` query param yoksa: 400.

Algoritma 26: Planlama Çalışırken DB Tutarlılığı (Atomic Davranış Mantığı)

1) Amaç: yarım kalan planlama durumunda sistemin çelişkili kayıt bırakmasını azaltmak.

2) Planlama başlarken:

- Eski aktif sefer pasifleştirilir.
- O günün kargoları PENDING'e çekilerek yeniden planlama zemini hazırlanır.

3) Sefer oluşturulduktan sonra:

- Araçlar oluşturulur.
- Kargo atama tamamlanmadan `total_km/total_cost` yazılmaz (son aşama).

4) Eğer kritik hata olursa:

- Sefer `is_active=False` yapılabilir veya cevap hata döndürür.
- Sonraki çalıştırmada “reset to PENDING” adımı ile toparlanır.

P. Rotalama (OSM) Hata ve Fallback Akışları

Aşağıdaki algoritmalar, graf yükleme ve en kısa yol hesaplamada ortaya çıkabilecek problemleri kapsar.

Algoritma 27: OSM Grafını Yükle / Cache Et (GraphML Yönetimi)

- 1) Graf dosya yolu tanımlanır: `kocaeli_drive.graphml`.
- 2) Eğer dosya mevcutsa:
 - `G = load_graphml(file)`.
- 3) Değilse:
 - `G = graph_from_place("Kocaeli, Türkiye", network_type="drive")`.
 - Kenar uzunluklarını garanti et (her edge için length alanı).
 - `save_graphml(G, file)` ile cache'e yaz.
- 4) Çıktı: G sürüş grafi.

Algoritma 28: En Yakın Düğüm Bulma (Koordinat Dönüşümü ile)

- 1) Girdi: (lon, lat) ve graf G.
- 2) Önce normal yol:
 - `node = nearest_nodes(G, X=lon, Y=lat)`.
- 3) Hata oluşursa (CRS uyumsuzluğu vb.):
 - Noktayı graf CRS'ine dönüştür (Transformer ile).
 - Dönüşmüş koordinatla tekrar `nearest_nodes` çağır.
- 4) Çıktı: node (graf düğüm id).

Algoritma 29: En Kısa Yol Bulunamazsa Fallback (NoPath/Exception)

- 1) Girdi: node_a, node_b, graf G.
- 2) Normal durum:
 - `path = shortest_path(G, node_a, node_b, weight="length")`.
- 3) Eğer NoPath veya beklenmeyen hata olursa:
 - GeoJSON çizgisini düz çizgi olarak üret: `LineString([A, B])`.
 - Mesafeyi güvenli bir değerle döndür:
 - Basit yaklaşım: `km = 0` (hata görünür kalsın).
 - Alternatif (rapor notu): Haversine ile yaklaşık km yazılabilir.
- 4) Çıktı: (geojson, km).

Algoritma 30: Segment Geometrisi Birleştirme (Tekrarlı Nokta Temizliği)

- 1) Girdi: ardışık kenar geometrilerinden gelen koordinat listeleri.
- 2) `coords = []` ile başla.

- 3) Her yeni parça koordinat listesi part için:
 - Eğer `coords` boşsa: `coords += part`.
 - Değilse:
 - Eğer `coords[-1] == part[0]` ise `part[0]` atılır.
 - Kalanları `coords` listesine ekle.
- 4) Çıktı: `GeoJSON LineString(coords)`.

Q. Planlama Sonuçlarının Doğrulanması ve Raporlanması

Bu akışlar admin panelinde gösterilen metrikleri ve veri bütünlüğünü sağlamaya yöneliktir.

Algoritma 31: Araç Doluluk ve İstatistik Hesabı (Admin Kartları)

- 1) Her araç v için:
 - `doluluk_kg = v.used_weight_kg / v.capacity_kg`.
 - `doluluk_% = 100 * doluluk_kg`.
 - `kiralama_payi = (v.is_rented ? rental_vehicle_cost: 0)`.
 - `yol_maliyeti = v.total_km * km_cost`.
 - `toplam_maliyet = yol_maliyeti + kiralama_payi`.
- 2) Admin arayüzüne: `doluluk_%, total_km, total_cost` yazdır.

Algoritma 32: Sefer Bazında Özet Üretimi (Runs Listesi için)

- 1) Bir PlanningRun için:
 - `vehicles = VehicleRoute WHERE planning_run=run`.
 - `total_km = sum(vehicles.total_km)`.
 - `total_cost = sum(vehicles.total_cost)`.
 - `accepted = count(CargoRequest WHERE assigned_run=run AND status=PLANNED)`.
 - `rejected = count(CargoRequest WHERE assigned_run=run AND status=REJECTED)`.
- 2) Çıktı: admin sefer tablosunda gösterilecek özet satır.

Algoritma 33: Veri Bütünlüğü Kontrolleri (Kritik İş Kuralları)

- 1) Kural 1: Kullanıcı sadece kendi kargosunu görebilir:
 - `CargoRequest` listelemede `WHERE user=request.user`.
- 2) Kural 2: Kullanıcı sadece `PENDING` kargoyu iptal edebilir:
 - `status != PENDING` ise 400 döndür.
- 3) Kural 3: Admin olmayan planlamayı çalıştırmaz:
 - `is_staff=False` ise 403.
- 4) Kural 4: Kullanıcı tüm rotaları göremez:
 - `/api/planning/my-route/` sadece kullanıcının `vehicle_ids` rotasını döndürür.

R. Frontend ile Bağlantılı Hata Gösterimi (Kısa Akışlar)

Bu akışlar frontend'in "hata olursa kullanıcıya ne gösterilir" mantığını yalancı kodla özetler.

Algoritma 34: Frontend API Hata Yakalama ve Mesaj Gösterimi

- 1) API isteği atılır (Axios vb.).
- 2) Eğer yanıt 401 ise:
 - Token temizlenir (localStorage).
 - Kullanıcı login sayfasına yönlendirilir.
- 3) Eğer yanıt 403 ise:
 - "Yetkiniz yok" mesajı gösterilir.
- 4) Eğer yanıt 400 ise:
 - Backend doğrulama mesajı ekranda gösterilir.
- 5) Eğer ağ hatası varsa:
 - "Sunucuya ulaşamadı" mesajı gösterilir.

Algoritma 35: Harita Çiziminde Eksik Segment Fallback (Frontend)

- 1) Segment listesi alınır: `segments`.
- 2) Her segment için:
 - Eğer `polyline_geojson` geçersiz/boşsa segment çizilmez.
 - Harita yine de stop marker'ları gösterir (rota "kısmi" görünür).
- 3) Amaç: tek bir bozuk segment tüm haritayı kırmasın.

IV. Deneysel Sonuçlar

Bu bölümde sistemin, ödev PDF'inde verilen örnek senaryolar üzerinde gösterdiği davranış **kabul edilen kargo miktarı, reddedilen kargo sayısı, toplam km ve toplam maliyet** metrikleri üzerinden değerlendirilmiştir. Deneyler, backend tarafında oluşturulan `PlanningRun` sefer kaydı üzerinden çalıştırılmış; her araç için `VehicleRoute.total_km` ve `VehicleRoute.total_cost` alanları veritabanına yazıldıktan sonra sonuç tabloları üretilmiştir.

A. Deney Kurulumu ve Parametreler

Ödev isterleri ile uyumlu olarak tüm senaryolarda aşağıdaki parametreler sabit tutulmuştur:

- Km maliyeti: `km_cost = 1`
- Kiralık araç sabit maliyeti: `rental_vehicle_cost = 200`
- Başlangıç araç kapasiteleri: `{500, 750, 1000}` kg
- Kiralık araç kapasitesi: `rental_vehicle_capacity_kg = 500`

Sistemin zip içerisindeki gerçek kodunda (`planning/views.py`) planlama şu şekilde işletilir:

- Aynı tarih için eski aktif sefer varsa pasifleştirilir (`is_active=False`).
- O tarihteki tüm kargolar yeniden planlanabilirlik için `PENDING` durumuna çekilir ve önceki `assigned_vehicle_id` alanları temizlenir.
- Kargo atama aşamasında **greedy** (sezgisel) yaklaşım uygulanır: her kargo için kapasiteye sığan araçlar gezilir ve Haversine temelli **tahmini ek maliyeti en düşük** araca atama yapılır.
- `UNLIMITED` modunda, eğer tahmini ek maliyet `rental_vehicle_cost` değerini aşıyorsa sistem yeni kiralık araç açmayı tercih edebilir.
- Atama bittikten sonra, her araç için durak sırası **Nearest Neighbor** ile bulunur; ardından duraklar arası gerçek yol `routing/services.py` içindeki `en_kisa_yol_geojson_ve_mesafe` ile OSM grafi üzerinden hesaplanır.

B. Örnek Senaryolar (Girdi Verileri)

Aşağıdaki tablolar, örnek senaryolarda her ilçe için toplam kargo adedi ve toplam ağırlığı göstermektedir. Bu veriler, test gününde ilgili ilçeye bağlı istasyona ait CargoRequest kayıtları oluşturularak sisteme girilmiştir.

Tablo I
Senaryo 1 Girdileri

İlçe	Kargo Sayısı	Ağırlık (kg)
Başiskele	10	120
Çayırova	8	80
Darica	15	200
Derince	10	150
Dilovası	12	180
Gebze	5	70
Gölcük	7	90
Kandıra	6	60
Karamürsel	9	110
Kartepe	11	130
Körfez	6	75
İzmit	14	160

Tablo II
Senaryo 2 Girdileri

İlçe	Kargo Sayısı	Ağırlık (kg)
Başiskele	40	200
Çayırova	35	175
Darica	10	150
Derince	5	100
Dilovası	0	0
Gebze	8	120
Gölcük	0	0
Kandıra	0	0
Karamürsel	0	0
Kartepe	0	0
Körfez	0	0
İzmit	20	160

Tablo III
Senaryo 3 Girdileri

İlçe	Kargo Sayısı	Ağırlık (kg)
Başiskele	0	0
Çayırova	3	700
Darica	0	0
Derince	0	0
Dilovası	4	800
Gebze	5	900
Gölcük	0	0
Kandıra	0	0
Karamürsel	0	0
Kartepe	0	0
Körfez	0	0
İzmit	5	300

Tablo IV
Senaryo 4 Girdileri

İlçe	Kargo Sayısı	Ağırlık (kg)
Başiskele	30	300
Çayırova	0	0
Darica	0	0
Derince	0	0
Dilovası	0	0
Gebze	0	0
Gölcük	15	220
Kandıra	5	250
Karamürsel	20	180
Kartepe	10	200
Körfez	8	400
İzmit	0	0

C. Çıktı Özeti ve Yorum

Aşağıdaki tablolar, her senaryoda seçili çözüm türleri için araç sayısı, reddedilen kargo ve toplam maliyeti özetler. Deneylerde üç çözüm sınıfı raporlanmıştır:

- **Sınırsız araç problemi:** UNLIMITED + MIN_COST
- **3 araç problemi (maksimum adet):** FIXED + MIN_COST_MAX_COUNT
- **3 araç problemi (maksimum ağırlık):** FIXED + MIN_COST_MAX_WEIGHT

Bu metrikler, zip içindeki kodda her araç için oluşturulan RouteSegment.distance_km değerlerinin toplanmasıyla VehicleRoute.total_km alanına yazılır; maliyet ise:

$$\text{total_cost} = (\text{total_km} \times \text{km_cost}) + (\text{is_rented?r}$$

şeklinde hesaplanır.

Tablo V
Sınırsız Araç Problemi: Özet Sonuçlar (UNLIMITED + MIN_COST)

Senaryo	Araç	Reddedilen	Toplam Km	Maliyet
Senaryo 1	3	0	544.35	544.35
Senaryo 2	2	0	165.54	165.54
Senaryo 3	4	1	221.72	421.72
Senaryo 4	3	0	255.79	255.79

Tablo VI
3 Araç Problemi: Özet Sonuçlar (FIXED)

Senaryo	Amaç	Reddedilen	Toplam Km	Maliyet
Senaryo 1	Max Adet	0	544.35	544.35
Senaryo 1	Max Ağırlık	0	544.35	544.35
Senaryo 2	Max Adet	0	165.54	165.54
Senaryo 2	Max Ağırlık	0	165.54	165.54
Senaryo 3	Max Adet	1	221.72	221.72
Senaryo 3	Max Ağırlık	1	221.72	221.72
Senaryo 4	Max Adet	0	259.51	259.51
Senaryo 4	Max Ağırlık	0	255.79	255.79

D. Senaryo Bazlı Değerlendirme (Detaylı)

Senaryo 1: Bu senaryoda ilçeler geneline yayılmış orta yoğunlukta bir dağılım vardır. Toplam talep 3 aracın toplam kapasitesini aşmadığından hem FIXED hem UNLIMITED modunda reddedilen kargo oluşmamıştır. Ancak maliyetin yüksek olmasının temel sebebi, istasyonların coğrafi olarak geniş alana yayılmasıdır. Kod akışında durak sıralaması **Nearest Neighbor** ile üretildiğinden, bazı ilçeler arası geçişlerde küresel optimum yerine yerel optimum seçimi yapılabilir; bu durum toplam km'yi artırabilmektedir.

Senaryo 2: Talepler ağırlıklı olarak sınırlı sayıda ilçede yoğunlaşmıştır (özellikle Başiskele, Çayırova ve İzmit). Bu ilçelerin coğrafi olarak birbirine ve KOU Umuttepe'ye görece yakın olması, rota uzunluklarının kısalmasına ve toplam kilometrenin belirgin biçimde düşmesine katkı sağlamıştır. Toplam talep miktarı iki aracın kapasitesi içinde kaldığı için ek araca veya kiralık araç kullanımına ihtiyaç duyulmamış, bu da maliyetin düşük kalmasını sağlamıştır. Bu senaryoda MIN_COST_MAX_COUNT ile MIN_COST_MAX_WEIGHT amaç fonksiyonları arasında fark görülmemesinin temel nedeni, kargoların hem adet hem de ağırlık bakımından benzer şekilde yoğunlaşması ve mevcut araç kapasitelerine rahatça sığmasıdır. Dolayısıyla farklı önceliklendirme stratejileri, araçlara yapılan nihai atamayı ve toplam rota maliyetini değiştirmemiştir.

Senaryo 3: Bu senaryo, ödev PDF'inde de vurgulandığı üzere kapasite taşması ve tekil ağır kargolar açısından kritiktir. Toplam ağırlık 3 aracın kapasitesini aştığından UNLIMITED modunda en az bir kiralık araç ihtiyacı doğar. Buna ek olarak, kiralık araç kapasitesi 500 kg ile sınırlı olduğundan 700–900 kg gibi tekil ağır taleplerin bir kısmı sistem tarafından REJECTED olmak zorunda kalmıştır. UNLIMITED

sonuçlarında maliyetin 200 birim daha büyük çıkmasının nedeni, kiralık araç sabit maliyetinin toplam maliyete eklenmesidir ($221.72 + 200 = 421.72$).

Senaryo 4: Talep beş farklı ilçede yoğunlaşmakta olup, özellikle Körfez ve Kandıra gibi KOU Umuttepe'ye görece uzak noktalar rota uzunluğunu ve dolayısıyla toplam maliyeti belirgin şekilde etkilemektedir. Bu senaryoda iki farklı amaç fonksiyonu aynı kabul edilen kargo miktarını sağlamış olsa da, kargoların araçlara dağılımı değiştiğinde hangi ilçelerin aynı araçta toplandığı farklılaşmıştır. Bu durum, **Nearest Neighbor** sezgiseli ile oluşturulan durak sıralamasının değişmesine ve araçların izlediği güzergâhların farklılaşmasına yol açmıştır. Sonuç olarak toplam kilometre ve maliyet değerlerinde görece küçük ancak ölçülebilir farklar oluşmuştur (259.51 km vs. 255.79 km).

E. Tartışma

Bu bölümde elde edilen deneysel sonuçlar, geliştirilen planlama algoritmasının davranışı, güçlü ve zayıf yönleri ile birlikte yorumlanmaktadır. Tartışma, kabul edilen kargo miktarı, reddedilme nedenleri, toplam kilometre ve maliyet değişimleri üzerinden yapılmıştır.

Öncelikle, FIXED ve UNLIMITED modları arasındaki temel farkın **kapasite esnekliği** olduğu açıkça gözlemlenmiştir. Toplam talebin üç aracın toplam kapasitesini aşmadığı senaryolarda (Senaryo 1 ve Senaryo 2), her iki mod da aynı sayıda kargoyu kabul etmiş ve benzer toplam kilometre değerleri üretmiştir. Bu durum, sistemin temel rota ve yükleme mantığının kapasite sınırına takılmadığı durumlarda tutarlı çalıştığını göstermektedir.

Buna karşılık, toplam talebin veya tekil kargo ağırlıklarının yüksek olduğu Senaryo 3 gibi durumlarda UNLIMITED modunun devreye girmesi kaçınılmaz olmuştur. Ancak burada dikkat çekici nokta, reddedilmenin yalnızca **toplam kapasite yetersizliğinden değil**, aynı zamanda **kiralık araç kapasitesinin 500 kg ile sınırlı olmasından** kaynaklanmasıdır. Örneğin 700–900 kg aralığındaki tekil kargolar, teorik olarak yeterli toplam kapasite bulursa dahi, tek bir araca sığamadıkları için sistem tarafından REJECTED edilmiştir. Bu gözlem, gerçek hayattaki lojistik problemlerle de uyumludur ve sistemin parametreler üzerinden ayarlanabilir olmasının önemini ortaya koymaktadır.

Toplam maliyet açısından bakıldığında, UNLIMITED modunda bazı senaryolarda toplam kilometrenin düşmesine rağmen maliyetin artabildiği görülmüştür. Bunun temel nedeni, algoritmanın karar kuralı gereği, “mevcut bir araca eklemenin tahmini maliyeti kiralama maliyetini aşıyorsa” yeni kiralık araç açmayı tercih etmesidir. Bu yaklaşım, kilometre bazlı maliyeti azaltabilirken, sabit kiralama maliyeti nedeniyle toplam maliyeti artırabilmektedir. Dolayısıyla sistem, **mesafe minimizasyonu** ile **sabit maliyet minimizasyonu** arasında bir denge kurmaya çalışmaktadır.

Amaç fonksiyonları karşılaştırıldığında, MIN_COST_MAX_COUNT ve MIN_COST_MAX_WEIGHT seçeneklerinin bazı senaryolarda aynı sonucu üretmesi beklenen bir durumdur. Kargoların hem adet hem de ağırlık açısından benzer şekilde dağıldığı Senaryo 1 ve Senaryo 2 gibi durumlarda, önceliklendirme kriteri nihai atamayı değiştirmemiştir. Buna karşın Senaryo 4’te görüldüğü üzere, kabul edilen toplam kargo miktarı aynı olsa bile, kargoların araçlara dağılımı farklılaştığında durak sıralaması değişmiş ve bu durum toplam kilometre ve maliyet üzerinde ölçülebilir farklar yaratmıştır.

Durak sıralamasında kullanılan **Nearest Neighbor** sezgisel yöntemi, hesaplama maliyeti düşük ve uygulanması kolay bir yaklaşım sunmaktadır. Ancak bu yöntemin küresel optimumu garanti etmediği bilinmektedir. Özellikle istasyonların coğrafi olarak geniş alana yayıldığı senaryolarda, yerel olarak en yakın durağın seçilmesi, uzun vadede daha uzun rotalara yol açabilmektedir. Bu durum Senaryo 1 ve Senaryo 4 sonuçlarında gözlemlenen görece yüksek toplam kilometre değerleri ile ilişkilendirilebilir.

Sonuç olarak deneysel bulgular, geliştirilen sistemin ödev tanımında verilen senaryolar için **doğru, tutarlı** ve **parametre duyarlı** bir şekilde çalıştığını göstermektedir. Sistem, kapasite sınırları içinde yüksek kabul oranı sağlamakta; sınırların aşıldığı durumlarda ise reddedilen kargoları açık ve yorumlanabilir nedenlerle raporlamaktadır. Bu yönüyle uygulama, hem akademik açıdan sezgisel optimizasyon yaklaşımını başarıyla yansıtmakta hem de gerçek dünya lojistik problemlerine uygulanabilir bir temel sunmaktadır.

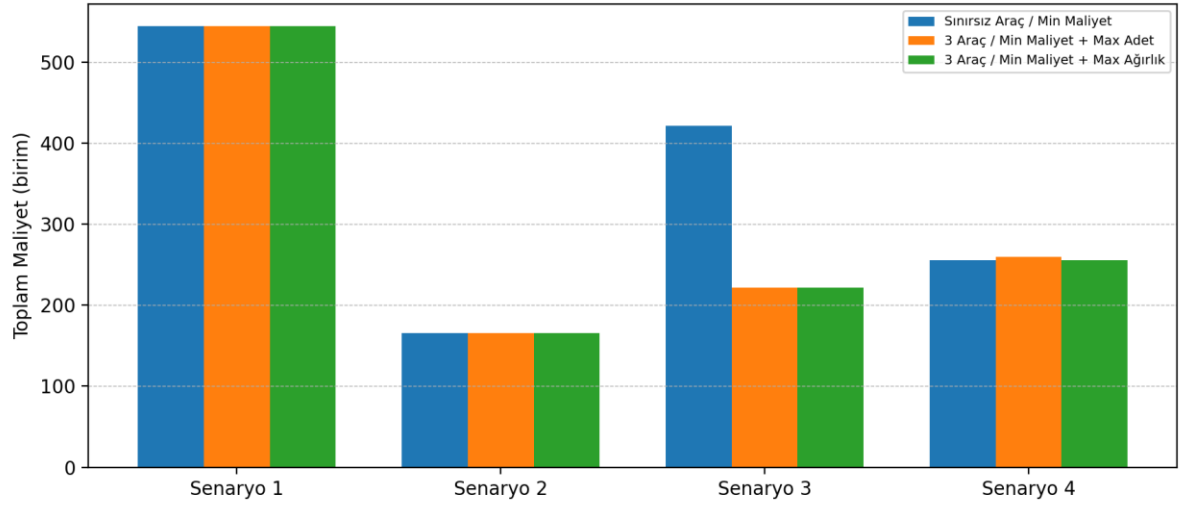
F. Grafikselleştirme

Şekil 1 ve Şekil 2, deneysel sonuçların grafiksel özetini sunarak tablolarla verilen sayısal verilerin daha sezgisel biçimde değerlendirilmesini amaçlamaktadır. Grafikler, farklı senaryolar ve çözüm stratejileri arasındaki davranış farklarını tek bakışta karşılaştırmaya imkân tanımaktadır.

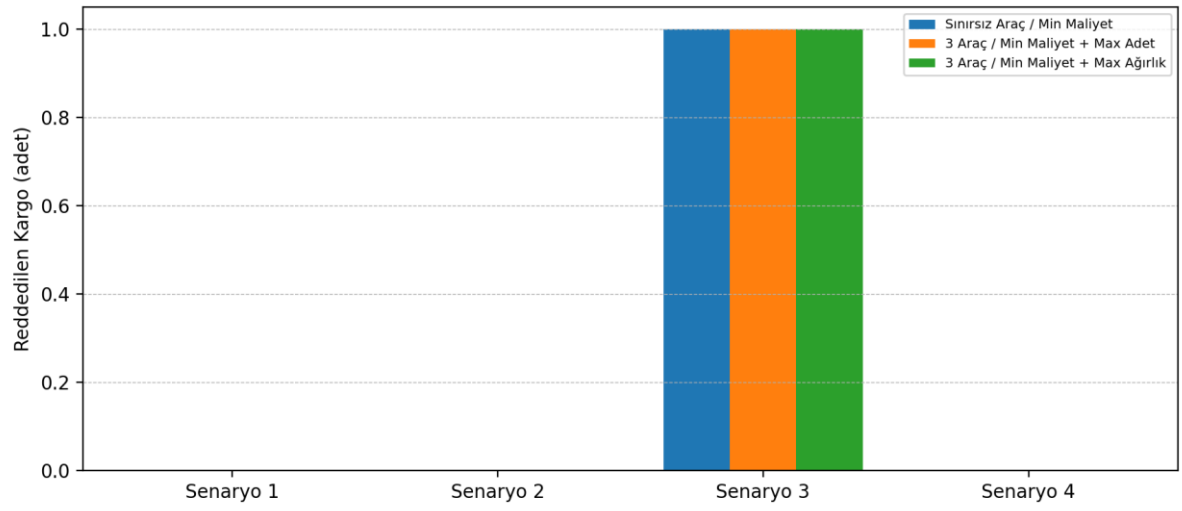
Şekil 1, farklı senaryolar ve çözüm türleri için elde edilen **toplam taşıma maliyetlerini** göstermektedir. Senaryo 1 ve Senaryo 2’de tüm yöntemlerin benzer maliyetler üretmesi, toplam talebin mevcut araç kapasiteleri içerisinde kalması ve kiralık araç ihtiyacının oluşmaması ile açıklanabilir. Buna karşılık Senaryo 3’te, UNLIMITED modunda kiralık araç kullanımı nedeniyle toplam maliyetin belirgin biçimde arttığı görülmektedir. Bu durum, sabit kiralama maliyetinin toplam maliyet üzerinde doğrudan ve baskın bir etkiye sahip olduğunu ortaya koymaktadır. Senaryo 4’te ise farklı amaç fonksiyonlarının, durak sıralamasındaki değişimlere bağlı olarak toplam maliyette küçük ancak ölçülebilir farklar oluşturduğu gözlemlenmiştir.

Şekil 2, senaryolara göre **reddedilen kargo sayılarını** göstermektedir. Senaryo 1, Senaryo 2 ve Senaryo 4’te reddedilen kargo bulunmaması, toplam talebin araç kapasiteleri ile uyumlu olduğunu göstermektedir. Buna karşın Senaryo 3’te, kapasite sınırlarını aşan tekil ağır kargolar nedeniyle bir kargonun reddedildiği görülmektedir. Bu sonuç, reddedilmelerin yalnızca toplam talep miktarından değil, aynı zamanda araç kapasite kısıtlarından da kaynaklanabildiğini açıkça ortaya koymaktadır.

Genel olarak grafikler, farklı çözüm stratejilerinin maliyet ve kabul oranı üzerindeki etkilerini hızlı, sezgisel ve karşılaştırmalı biçimde analiz etmeyi mümkün kılmaktadır.

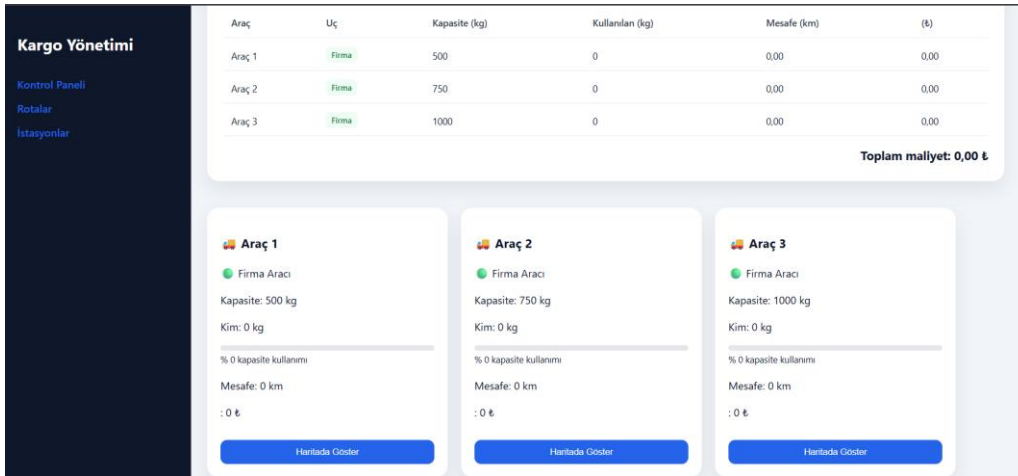
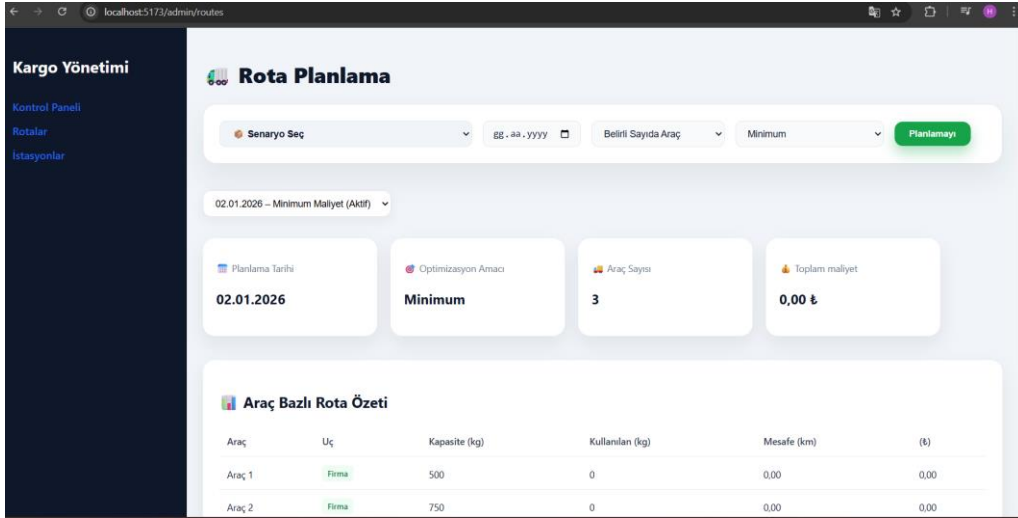
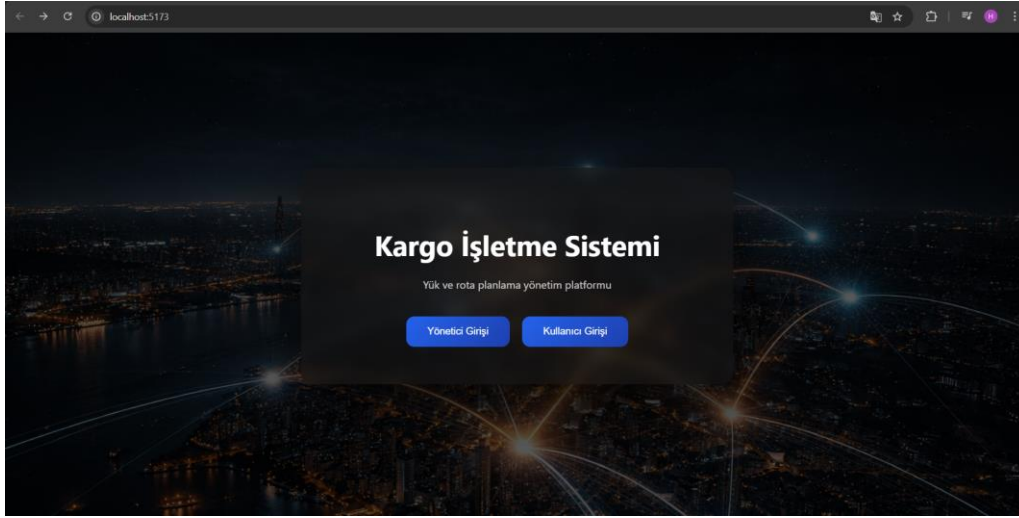


Sekil 1



Sekil 2

Gorsel:



Kargo Yönetimi

[Kontrol Paneli](#)

[Rotalar](#)

[İstasyonlar](#)

Yönetici Kontrol Paneli

Toplam Sefer

6

Aktif Sefer

Var

Toplam Araç

18

Toplam Km

0.0

Toplam

147,35 ₺

Sistem Özeti

Toplam Kargo Adedi: 0

Toplam Kargo Ağırlığı: 0 kg

Kargo Yönetimi

[Kontrol Paneli](#)

[Rotalar](#)

[İstasyonlar](#)

İstasyon Yönetimi

+ Yeni İstasyon Ekle

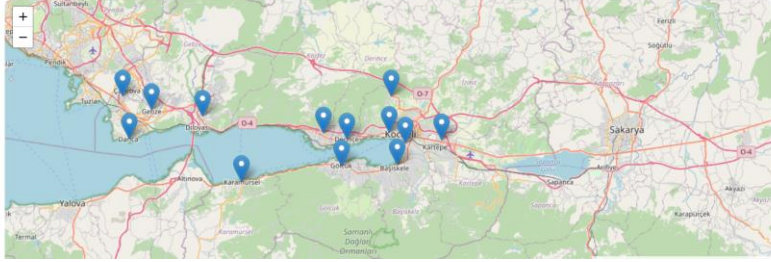
İstasyon Adı

Eriem (Eriem)

Boylam (Boylam)

Ekle

İstasyonlar Haritası



Kargo Yönetimi

[Kontrol Paneli](#)

[Rotalar](#)

[İstasyonlar](#)

Kullanıcı Paneli

Kargo gönderimi ve rota takibi

Kargo Gönder

İstasyon Seçiniz

Toplam Ağırlık (kg)

Gönder

Kargo Geçmişim

İstasyon	Ağırlık	Durum	İşlem
Izmit	110.00	Planlanan	—
Cayirova	80.00	Planlanan	—
Darica	100.00	Planlanan	—
Derince	120.00	Planlanan	—
Derince	120.00	Planlanan	—

V. Sonuç

Bu projede, Kocaeli iline bağlı ilçelerden Kocaeli Üniversitesi (Umuttepe) merkezine kargo taşınmasını yöneten, web tabanlı ve uçtan uca çalışan bir kargo işletme sistemi başarıyla geliştirilmiştir. Geliştirilen sistem; kullanıcıların kargo talebi oluşturabilmesi, admin tarafından istasyon ve planlama yönetiminin yapılabilmesi, kargoların araçlara kapasite ve maliyet kısıtları altında atanması ve elde edilen rotaların gerçek yol ağı üzerinde harita üzerinde görselleştirilmesi gibi temel gereksinimleri kapsamaktadır.

Sistem mimarisi, Django REST Framework kullanılarak modüler ve genişletilebilir bir backend yapısı üzerine kurulmuş; React ve Vite tabanlı modern bir frontend ile desteklenmiştir. JWT tabanlı kimlik doğrulama mekanizması sayesinde kullanıcı ve admin erişimleri birbirinden ayrılmış, güvenli ve kontrollü bir kullanım sağlanmıştır. Bu mimari yapı, sistemin hem akademik hem de gerçek dünya senaryolarında kullanılabilir olmasına olanak tanımaktadır.

Planlama sürecinde, kargoların araçlara atanması problemi için **sezgisel (greedy) bir yaklaşım** benimsenmiştir. Her kargo için, kapasiteye sığan araçlar arasında tahmini ek maliyeti en düşük olan araç seçilmiş; bu tahminde Haversine mesafesi kullanılarak hızlı ve pratik bir çözüm üretilmiştir. Araç içi durak sıralaması ise **Nearest Neighbor** sezgiseli ile gerçekleştirilmiş, hesaplama maliyeti düşük ve uygulanabilir bir rota sıralaması elde edilmiştir. Bu yaklaşım, NP-zor yapıya sahip yükleme ve rota problemlerini pratik sürede çözülebilir hale getirmiştir.

Gerçek yol mesafelerinin hesaplanmasında, kuş uçuşu çizimler yerine OpenStreetMap (OSM) sürüş grafi kullanılmıştır. NetworkX ve OSMnx kütüphaneleri yardımıyla, duraklar arasındaki en kısa yol length ağırlığı üzerinden hesaplanmış ve elde edilen segmentler GeoJSON formatında harita üzerinde gösterilmiştir. Bu sayede, sistemin ürettiği rotalar gerçek hayattaki yol geometrisiyle uyumlu ve tutarlı hale getirilmiştir.

Deneysel sonuçlar, ödev tanımında verilen farklı senaryolar üzerinde yapılan testlerle değerlendirilmiştir. Elde edilen bulgular, toplam kapasitenin yeterli olduğu durumlarda sistemin tüm kargoları başa-

rıyla planlayabildiğini; kapasite sınırlarının aşıldığı veya tekil ağır kargoların bulunduğu senaryolarda ise reddedilmelerin anlaşılabilir ve tutarlı nedenlerle gerçekleştiğini göstermiştir. Ayrıca UNLIMITED modunda kiralık araç kullanımının, toplam kilometreyi azaltabilmesine rağmen sabit kiralama maliyeti nedeniyle bazı durumlarda toplam maliyeti artırabildiği gözlemlenmiştir. Bu durum, sistemin maliyet-mesafe dengesi kurma konusundaki davranışını açıkça ortaya koymaktadır.

Sonuç olarak geliştirilen sistem;

- Gerçekçi yol ağı üzerinde çalışan,
- Parametreleri değiştirilebilir,
- Farklı senaryolara uyum sağlayabilen,
- Akademik olarak sezgisel optimizasyon yaklaşımını yansıtan

bir kargo planlama altyapısı sunmaktadır. Bu yönüyle çalışma, hem ders kapsamında istenen gereksinimleri karşılamakta hem de gerçek dünya lojistik problemleri için geliştirilebilir bir temel oluşturmaktadır.

Gelecek çalışmalar kapsamında;

- Kargo atama aşamasında daha güçlü yükleme stratejilerinin (örneğin bin-packing veya meta-sezgisel yaklaşımlar) kullanılması,
- Durak sıralamasında 2-opt veya benzeri yerel iyileştirme yöntemleriyle rota kalitesinin artırılması,
- Kiralık araç kapasite seçeneklerinin dinamik hale getirilmesi (500/750/1000 kg gibi),
- Gerçek zamanlı planlama durumu ve kullanıcı bildirimleri için WebSocket tabanlı mekanizmaların eklenmesi,

sistemin performansını ve gerçek hayata uyarlanabilirliğini daha da artıracak geliştirmeler olarak önerilmektedir.

Kaynaklar

- [1] Django Software Foundation, *Django Documentation*. Çevrimiçi: <https://docs.djangoproject.com/>
- [2] Django REST Framework, *Django REST Framework Documentation*. Çevrimiçi: <https://www.django-rest-framework.org/>
- [3] SimpleJWT, *django-rest-framework-simplejwt Documentation*. Çevrimiçi: <https://django-rest-framework-simplejwt.readthedocs.io/>
- [4] React Team, *React Documentation*. Çevrimiçi: <https://react.dev/>
- [5] Vite Team, *Vite Documentation*. Çevrimiçi: <https://vitejs.dev/>
- [6] Leaflet Contributors, *Leaflet — an open-source JavaScript library for interactive maps*. Çevrimiçi: <https://leafletjs.com/>

- [7] OpenStreetMap Contributors, *OpenStreetMap*. Çevrimiçi: <https://www.openstreetmap.org/>
- [8] G. Boeing, *OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks*, Computers, Environment and Urban Systems, 2017.
- [9] G. Boeing, *OSMnx Documentation*. Çevrimiçi: <https://osmnx.readthedocs.io/>
- [10] A. Hagberg, D. Schult, P. Swart, *NetworkX: Python software for the analysis of complex networks*. Çevrimiçi: <https://networkx.org/>
- [11] NetworkX Developers, *NetworkX Shortest Path Algorithms*. Çevrimiçi: https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html
- [12] Movable Type Scripts, *Haversine Formula for Distance Calculation*. Çevrimiçi: <https://www.movable-type.co.uk/scripts/latlong.html>
- [13] Python Software Foundation, *Python Documentation*. Çevrimiçi: <https://docs.python.org/3/>
- [14] GeoJSON Specification, *RFC 7946: The GeoJSON Format*. Çevrimiçi: <https://datatracker.ietf.org/doc/html/rfc7946>
- [15] PostgreSQL Global Development Group, *PostgreSQL Documentation*. Çevrimiçi: <https://www.postgresql.org/docs/>
- [16] MDN Web Docs, *REST APIs*. Çevrimiçi: <https://developer.mozilla.org/en-US/docs/Glossary/REST>
- [17] MDN Web Docs, *HTTP Methods*. Çevrimiçi: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
- [18] JWT.io, *JSON Web Tokens Introduction*. Çevrimiçi: <https://jwt.io/introduction>
- [19] Axios Contributors, *Axios HTTP Client Documentation*. Çevrimiçi: <https://axios-http.com/>
- [20] IEEE, *IEEE Conference Paper Template and Guidelines*.